

**[SMART EXAMINATION INVIGILATION SYSTEM (SEIS) WITH AI
INTEGRATION]**

[AL-ASRY BIN AL-AMEEN]

UNIVERSITI TEKNIKAL MALAYSIA MELAKA

BORANG PENGESAHAN STATUS LAPORAN

JUDUL: [SMART EXAMINATION INVIGILATION SYSTEM (SEIS) WITH AI INTEGRATION]

SESI PENGAJIAN: [2025 / 2026]

Saya: [AL-ASRY BIN AL-AMEEN]

mengaku membenarkan tesis Projek Sarjana Muda ini disimpan di Perpustakaan Universiti Teknikal Malaysia Melaka dengan syarat-syarat kegunaan seperti berikut:

1. Tesis dan projek adalah hakmilik Universiti Teknikal Malaysia Melaka.
2. Perpustakaan Fakulti Kecerdasan Buatan dan Keselamatan Siber dibenarkan membuat salinan untuk tujuan pengajian sahaja.
3. Perpustakaan Fakulti Kecerdasan Buatan dan Keselamatan Siber dibenarkan membuat salinan tesis ini sebagai bahan pertukaran antara institusi pengajian tinggi.
4. Sila tandakan (✓)

SULIT

(Mengandungi maklumat yang berdarjah keselamatan atau kepentingan Malaysia seperti yang termaktub di dalam AKTA RAHSIA RASMI 1972)

TERHAD

(Mengandungi maklumat TERHAD yang telah ditentukan oleh organisasi / badan di mana penyelidikan dijalankan)

TIDAK TERHAD

(TANDATANGAN PELAJAR)

(TANDATANGAN PENYELIA)

Alamat tetap: _____

Nama Penyelia

Tarikh:

Tarikh:

CATATAN: Jika tesis ini SULIT atau TERHAD, sila lampirkan surat daripada pihak berkuasa.

[SMART EXAMINATION INVIGILATION SYSTEM (SEIS) WITH AI
INTEGRATION]

[AL-ASRY BIN AL-AMEEN]

This report is submitted in partial fulfillment of the requirements for the
Bachelor of [Computer Science (Artificial Intelligence)] with Honours.

FACULTY OF ARTIFICIAL INTELLIGENCE AND CYBER SECURITY
UNIVERSITI TEKNIKAL MALAYSIA MELAKA

[YEAR OF SUBMISSION]

DECLARATION

I hereby declare that this project report entitled
**[SMART EXAMINATION INVIGILATION SYSTEM (SEIS) WITH AI
INTEGRATION]**
is written by me and is my own effort and that no part has been plagiarized
without citations.

STUDENT : _____ Date : _____
(AL-ASRY BIN AL-AMEEN)

I hereby declare that I have read this project report and found
this project report is sufficient in term of the scope and quality for the award of
Bachelor of [Computer Science (Artificial Intelligence)] with Honours.

SUPERVISOR : _____ Date : _____
([DR. KHARISMI BIN BURHANUDIN])

DEDICATION

[To my beloved parents, Azizah bt VSS Abdul and Al Ameen bin Abdul Hassan, for their endless support, patience, and motivation throughout my academic journey]

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to my supervisor, DR. KHARISMI BIN BURHANUDIN for their invaluable guidance, feedback, and encouragement in completing this project successfully. I would also like to extend a special thanks to my friend, who generously volunteered their time to help me rigorously test the AI tracking system and its various behavioral detection thresholds.

ABSTRACT

Academic dishonesty during physical examinations in large halls is a significant challenge for educational institutions. Manual invigilation is limited by human capacity, making it difficult for a small number of proctors to monitor many students effectively. This project proposes an AI Exam Invigilator System designed to supplement the physical proctoring process using computer vision and deep learning techniques. The system utilizes IP cameras or localized webcams to detect suspicious activities in real-time. It employs the RF-DETR (Real-time Detection Transformer) model combined with ByteTrack to detect and track prohibited items such as concealed mobile phones. Furthermore, the system tracks the student's head pose and eye gaze using MediaPipe Face Mesh to identify periods of abnormal gaze deviation, such as attempting to look at another student's paper. When a violation is detected, a screenshot (for objects) or a short video clip (for behaviors) is automatically recorded for evidence. An integrated Flask-based admin dashboard provides a centralized interface for invigilators to view live statistics, configure sensitivity thresholds, and review logged violations. The results demonstrate that the system can accurately detect prohibited items and monitor student attention, providing a robust solution to maintain academic integrity in physical examination settings.

ABSTRAK

Ketidakjujuran akademik semasa peperiksaan fizikal di dewan peperiksaan merupakan satu cabaran penting bagi institusi pendidikan. Pengawasan secara manual dibatasi oleh kapasiti manusia, menjadikannya sukar untuk bilangan pengawas yang kecil memantau ramai pelajar dengan berkesan. Projek ini mencadangkan Sistem Pengawas Peperiksaan AI yang direka untuk menyokong proses pengawasan fizikal menggunakan penglihatan komputer dan teknik pembelajaran mendalam. Sistem ini menggunakan suapan kamera IP atau kamera web untuk mengesan aktiviti yang mencurigakan dalam masa nyata. Ia menggunakan model RF-DETR yang digabungkan dengan ByteTrack untuk mengesan dan menjejaki barang terlarang seperti telefon bimbit yang disembunyikan. Selain itu, sistem ini menjejaki pergerakan kepala dan renungan mata pelajar menggunakan MediaPipe Face Mesh untuk mengenal pasti penyimpangan renungan yang tidak normal, seperti percubaan melihat kertas pelajar lain. Apabila pelanggaran dikesan, tangkapan skrin (untuk objek) atau klip video (untuk tingkah laku) akan direkod secara automatik sebagai bukti. Papan pemuka pentadbir berasaskan Flask bersepadu menyediakan antara muka berpusat kepada pengawas untuk melihat statistik langsung, mengkonfigurasi tahap sensitiviti, dan menyemak rekod pelanggaran. Keputusan menunjukkan bahawa sistem ini dapat mengesan barang terlarang dan memantau perhatian pelajar dengan tepat, sekali gus menyediakan penyelesaian teguh untuk mengekalkan integriti akademik dalam persekitaran peperiksaan fizikal.

TABLE OF CONTENTS

	PAGE
DECLARATION.....	II
DEDICATION.....	III
ACKNOWLEDGEMENTS.....	IV
ABSTRACT	V
ABSTRAK	VI
TABLE OF CONTENTS.....	VII
LIST OF TABLES	X
LIST OF FIGURES	XI
LIST OF ABBREVIATIONS	XII
LIST OF ATTACHMENTS.....	XIII
CHAPTER 1: INTRODUCTION.....	1
1.1 Introduction.....	1
1.2 Problem Statement	1
1.3 Objective	1
1.4 Scope.....	2
1.5 Project Significance	2
1.6 Expected Output.....	2
1.7 Report Organisation	3

1.8	Summary	3
CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY .		4
2.1	Introduction.....	4
2.2	Facts and Findings	4
2.2.1	Domain	4
2.2.2	Existing Systems.....	4
2.2.3	Technique	5
2.3	Project Methodology.....	5
2.4	Project Requirements	5
2.4.1	Software Requirements.....	5
2.4.2	Hardware Requirements	6
2.4.3	Other Requirements	6
2.5	Project Schedule and Milestones	7
2.6	Summary	7
CHAPTER 3: REQUIREMENT ANALYSIS		8
3.1	Introduction.....	8
3.2	Problem Analysis	8
3.3	Requirement Analysis	11
3.3.1	Data Requirement	11
3.3.2	Functional Requirement.....	12
3.3.3	Non-Functional Requirement	12
3.3.4	Other Requirement.....	13
3.4	Summary	13
CHAPTER 4: DESIGN		14

4.1	Introduction.....	14
4.2	High-Level Design.....	14
4.2.1	System Architecture.....	14
4.2.2	User Interface Design	16
4.2.3	Database Design	19
4.3	AI Component Design	20
4.4	Software or Hardware Design.....	22
4.5	Summary	22
CHAPTER 5: RESULTS AND DISCUSSION		24
5.1	Introduction.....	24
5.2	Evaluation of AI Techniques used in the project.....	24
5.3	Testing of functional Requirement	25
5.4	Summary	29
CHAPTER 6: CONCLUSION.....		30
6.1	Observation on Weakness and Strengths	30
6.2	Propositions for Improvement	30
6.3	Project Contribution.....	30
6.4	Summary	31
REFERENCES.....		32
APPENDICES		33

LIST OF TABLES**PAGE**

Table 3.1: Table Example.....	8
--------------------------------------	----------

LIST OF FIGURES

PAGE

Figure 2.1: Caption Window	Error! Bookmark not defined.
Figure 2.2: Caption Numbering Window	Error! Bookmark not defined.
Figure 2.3: Example Figure with citation from a book (Pike, 2008. Used with permission).....	Error! Bookmark not defined.
Figure 4.1: How to update Table of Contents in this template	Error! Bookmark not defined.
Figure 4.2: Updating the whole list.....	Error! Bookmark not defined.

LIST OF ABBREVIATIONS

FYP - **Final Year Project**

LIST OF ATTACHMENTS

		PAGE
Appendix A	Sample of data	19
Appendix B	Analysis of data collection	78
.....		
.....		

CHAPTER 1: INTRODUCTION

1.1 Introduction

The landscape of educational assessments heavily relies on examinations to evaluate student knowledge. However, the integrity of these examinations is constantly challenged by various forms of cheating. This project introduces the AI Exam Invigilator System, an automated computer vision application designed to assist human proctors in physical examination halls.

1.2 Problem Statement

Maintaining academic integrity during physical examinations in large halls remains a significant challenge for educational institutions. A limited number of human invigilators are often responsible for monitoring a large group of candidates, making it difficult to detect subtle forms of cheating, such as concealing mobile phones under desks or peering at adjacent student's papers. Relying entirely on manual supervision is inefficient and prone to human error or fatigue when attempting to monitor multiple students simultaneously over an extended period. Consequently, there is a critical need for an automated, intelligent camera-based system that can supplement physical proctors by continuously monitoring test-takers and accurately flagging suspicious behaviors or prohibited objects for human review.

1.3 Objective

- To develop an AI-powered exam invigilation system utilizing object detection (RF-DETR) to identify prohibited items such as mobile phones

- To implement a real-time tracking module that monitors the student's head pose and eye gaze (via MediaPipe) to detect inattention or suspicious looking behaviors.
- To design and integrate a web-based admin dashboard using Flask that logs violations, records automated video evidence, and allows invigilators to configure system parameters dynamically.

1.4 Scope

- Target Environment: Physical examination halls monitored via webcams or RTSP IP cameras.
- Technology Stack: Python for the core logic, RF-DETR for object detection, MediaPipe for facial landmarks, and Flask for the admin dashboard.
- Core Functionalities: Detection of objects (phones, books, laptops), head pose estimation (pitch, yaw), gaze tracking, and automated screenshot and video clip recording.

1.5 Project Significance

This project significantly benefits educational institutions by providing an extra layer of security during examinations. Invigilators can focus their attention on responding to high-confidence alerts generated by the system rather than constantly surveying the entire hall. This reduces proctor fatigue and helps maintain a fair testing environment.

1.6 Expected Output

The expected output is a fully functional, real-time desktop application and an accompanying web dashboard. The system will successfully process live camera feeds, highlight violations on a Heads-Up Display (HUD), save violation screenshots and video clips locally, and serve this data seamlessly to the admin dashboard.

Report Organization

1.7 Report Organisation

Chapter 1: Introduction – Discusses the project background, problem statement, objectives, and scope.

Chapter 2: Literature Review and Project Methodology – Reviews existing invigilation systems, AI techniques (RF-DETR, MediaPipe), and outlines the Prototyping methodology used.

Chapter 3: Requirement Analysis – Details the hardware, software, functional, and non-functional requirements.

Chapter 4: Design – Explains the high-level architecture, user interfaces, database schema, and AI component structure.

Chapter 5: Results and Discussion – Presents the testing strategies, AI evaluation metrics, and functional test results.

Chapter 6: Conclusion – Summarizes project strengths, weaknesses, and potential future improvements.

1.8 Summary

This chapter laid the foundational context for the AI Exam Invigilator System, outlining the core problems in physical exam monitoring and defining the objectives and scope of the proposed AI-based solution. The next chapter will explore existing systems and the methodology chosen for development.

CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY

2.1 Introduction

This chapter reviews related research in automated exam proctoring and the specific computer vision models utilized. It also defines the software development methodology chosen for the project.

2.2 Facts and Findings

2.2.1 Domain

The domain of this project is Artificial Intelligence, specifically Computer Vision. Computer vision allows machines to extract high-dimensional data from the real world (via cameras) to produce numerical or symbolic information. In the context of exam proctoring, it involves real-time object detection and facial feature tracking.

2.2.2 Existing Systems

Currently, most institutions rely heavily on pure manual invigilation in physical halls. Some modern remote proctoring systems (like ProctorU or Respondus) exist, but they are designed for students taking exams at home on personal computers, requiring screen locking and browser control. This project differs as it is designed for a localized, physical setting utilizing external cameras (CCTV or localized webcams) to track physical objects and candidate head movements.

2.2.3 Technique

- Object Detection: Traditional systems use YOLO variants. However, this project utilizes RF-DETR (Real-time Detection Transformer), which eliminates the need for Non-Maximum Suppression (NMS) and provides competitive real-time accuracy.
- Face Tracking: MediaPipe Face Mesh is utilized over standard Haar Cascades because it provides 468 precise 3D facial landmarks in real-time, allowing for accurate calculation of head yaw and pitch, , as well as computing iris locations for precise eye tracking (gaze detection).

2.3 Project Methodology

This project adopts Prototyping Methodology.

1. Requirement Gathering: Identifying the need for phone detection and gaze tracking based on common physical exam cheating methods.
2. Quick Design: Designing the detection loop (`main.py`) and the Flask dashboard (`admin_server.py`) as separate but communicating entities.
3. Building Prototype: Implementing RF-DETR and MediaPipe within an OpenCV frame-reading loop.
4. Customer Evaluation (Testing): Testing the system with different camera angles and adjusting confidence thresholds.
5. Refining: Adding the automated evidence recording modules (screenshots and video clips) based on the need for post-exam evidence.

2.4 Project Requirements

2.4.1 Software Requirements

- Python 3.9+

- ``opencv-python`` (Camera and frame rendering)
- ``rfdetr`` & ``supervision`` (Object detection and ByteTrack tracking)
- ``mediapipe`` (Facial landmarks)
- ``flask`` (Web dashboard)
- SQLite3 (Database)

2.4.2 Hardware Requirements

- Multi-core CPU (GPU recommended for faster RF-DETR inference).
- USB Webcam or IP Camera (RTSP compatible).
- Minimum 8GB RAM.

2.4.3 Other Requirements

- Adequate lighting in the examination hall for optimal camera performance.

2.5 Project Schedule and Milestones

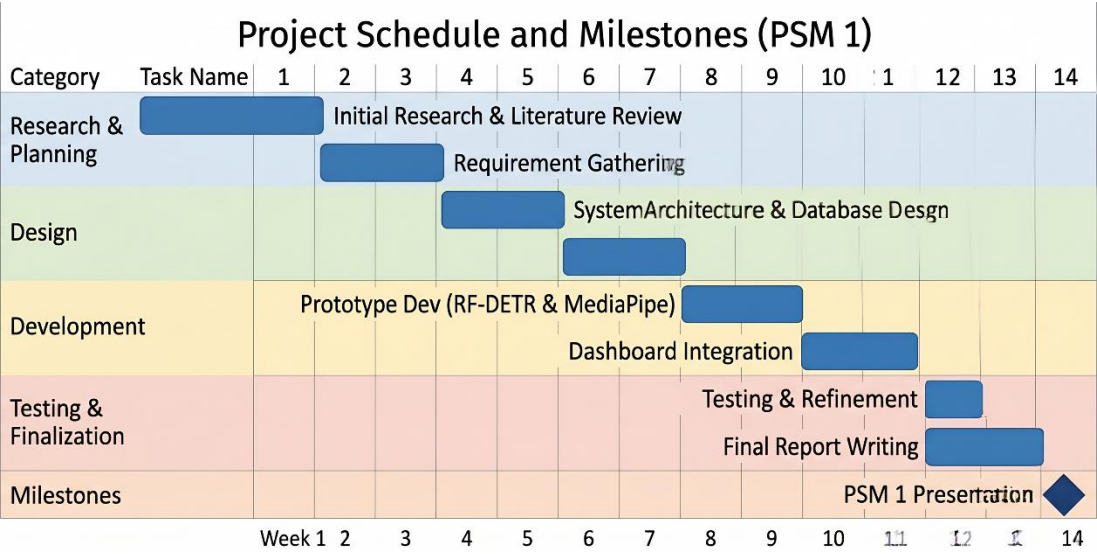


Figure 2.1: Gantt Chart

2.6 Summary

This chapter justified the selection of RF-DETR and MediaPipe over older techniques and outlined the Prototyping methodology and system requirements. The subsequent chapter will analyze the specific requirements of the proposed system.

CHAPTER 3: REQUIREMENT ANALYSIS

3.1 Introduction

This chapter details the specific problem scenarios currently faced during exams and translates them into concrete data and functional requirements for the proposed AI system.

3.2 Problem Analysis

In the current manual invigilation system, human proctors are physically present in the examination hall to monitor students. The standard operating procedure involves invigilators patrolling the aisles and observing students' behaviors. However, this method is highly dependent on the invigilator's line of sight and attention span. A student intending to cheat often waits for the invigilator's back to turn before accessing a concealed mobile phone or looking at unauthorized materials. Because human attention cannot be divided perfectly across multiple targets, many brief instances of academic dishonesty go undetected. Requirement Analysis

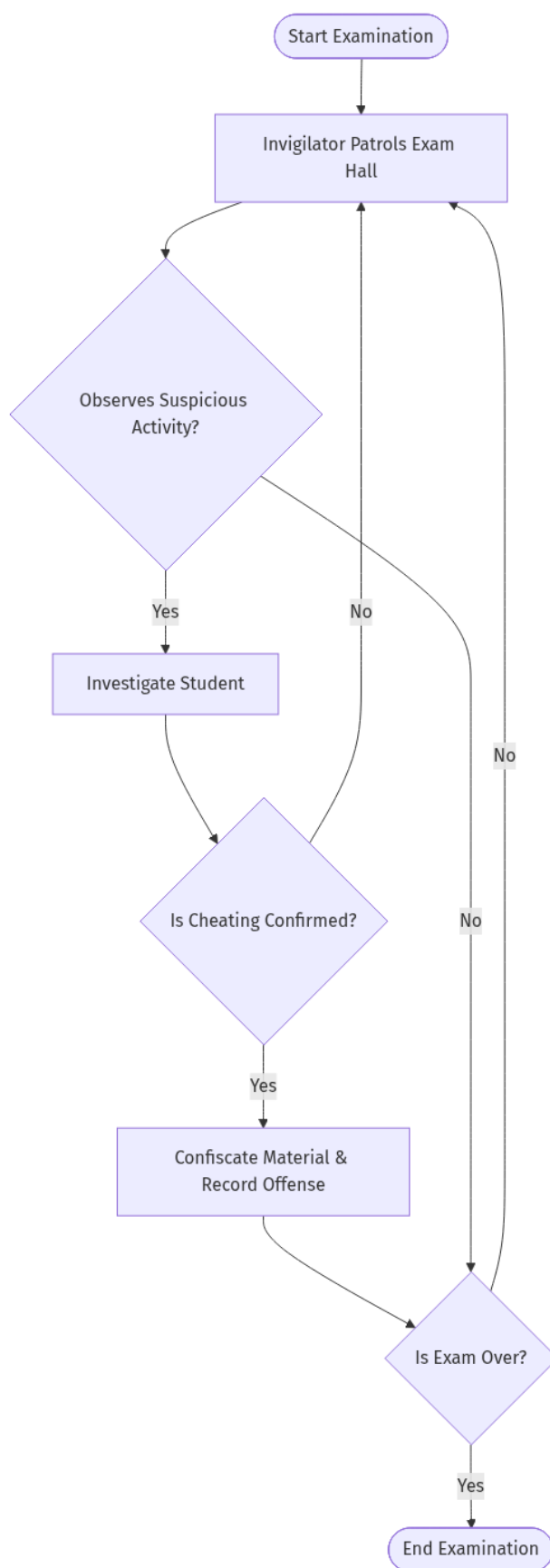


Figure 3.1: Traditional Human Invigilation

In a manual system, the probability of detecting a cheating event is low. For example, if an exam hall has 100 students and 2 invigilators, and each invigilator can effectively monitor 5 students at any given moment, the probability that a specific student is being watched is calculated as:

$$(2 \text{ invigilators} \times 5 \text{ students}) / 100 \text{ total students} = 0.10 \text{ (or 10 percent).}$$

To solve this, the proposed computational model uses continuous video analysis. The system validates violations mathematically using two main parameters to prevent false alarms:

1. Confidence Score: The AI model must be at least 75 percent certain that an object is a phone (Confidence > 0.75).
2. Time Duration: The object must be present continuously for more than 0.5 seconds, while an abnormal head pose or gaze deviation must persist for more than 5 seconds.

Only when both computational conditions are met does the system mathematically validate the event as a true violation.

The process to solve the problem involves:

1. Continuous Data Acquisition: Localized cameras capture a continuous stream of video frames (15 to 30 frames per second), ensuring 100 percent monitoring coverage of the camera's view.
2. Computational Analysis: Each frame is processed computationally by the RF-DETR model to identify prohibited items. Simultaneously, MediaPipe calculates the 3D geometry of the face to compute the yaw and pitch angles of the head, and extracts iris landmarks for gaze detection.

3. Algorithmic Validation: The system applies the mathematical rules mentioned above (checking confidence and time duration) to validate if a violation occurred.

4. Automated Logging: Once validated, the system bypasses human intervention by automatically recording a screenshot or video clip and saving the event into the database for the admin dashboard.

3.3 Requirement Analysis

3.3.1 Data Requirement

The system requires real-time video frames (RGB format) as input. The data generated by the system is stored in an SQLite database ('violations.db'). The Data Dictionary for the database is as follows:

Table 3.1

Field Name	Data Type	Description
'id'	INTEGER	Primary Key. Auto-incremented unique identifier for each violation.
'timestamp'	DATETIME	The exact date and time when the violation occurred.
'type'	VARCHAR	The category of the violation (e.g., 'phone', 'eye_gaze', 'head_pose')
'confidence'	FLOAT	The AI model's confidence score or deviation magnitude.
'video_path'	VARCHAR	The local file path pointing to the recorded evidence ('.jpg' screenshot or '.mp4' video).

3.3.2 Functional Requirement

- **Object Detection Module:** Must detect cell phones with a configurable confidence threshold.
- **Head Pose Module:** Calculates the yaw and pitch of the student's face to trigger an alert if the head rotation exceeds a defined threshold for a continuous duration.
- **Eye Tracking Module:** Tracks the student's iris relative to eye landmarks to detect continuous, abnormal gaze deviations (e.g., looking sideways).
- **Evidence Recording:** Saves a screenshot for object detection violations and automatically saves a 3-second video clip locally when a behavioral rule is broken.
- **Admin Dashboard:** Must serve a web page that fetches violation records from the database and allows video playback.

The functional flow and boundaries of the system are illustrated in the Context Diagram below:

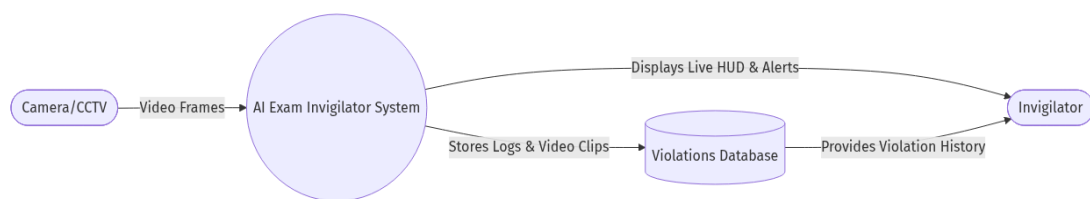


Figure 3.2: Context Diagram

3.3.3 Non-Functional Requirement

- **Performance:** The detection loop should run at a minimum of 15-20 FPS to ensure accurate real-time tracking without lagging.

- Reliability: The system must run continuously for the duration of an exam (e.g., 2-3 hours) without crashing.
- Usability: The web dashboard must be intuitive for non-technical invigilators to navigate.

3.3.4 Other Requirement

Privacy considerations must be managed by ensuring video clips are stored securely locally and deleted after the exam review period.

3.4 Summary

This chapter established the functional and non-functional requirements necessary to build a reliable and robust invigilation system. The next chapter will cover the architectural design of these components.

CHAPTER 4: DESIGN

4.1 Introduction

This chapter defines the system architecture, user interface design, and the integration of the AI components into the overall software workflow.

4.2 High-Level Design

4.2.1 System Architecture

The system follows a layered architecture to process visual data and output administrative reports. The architecture view is presented by layer:

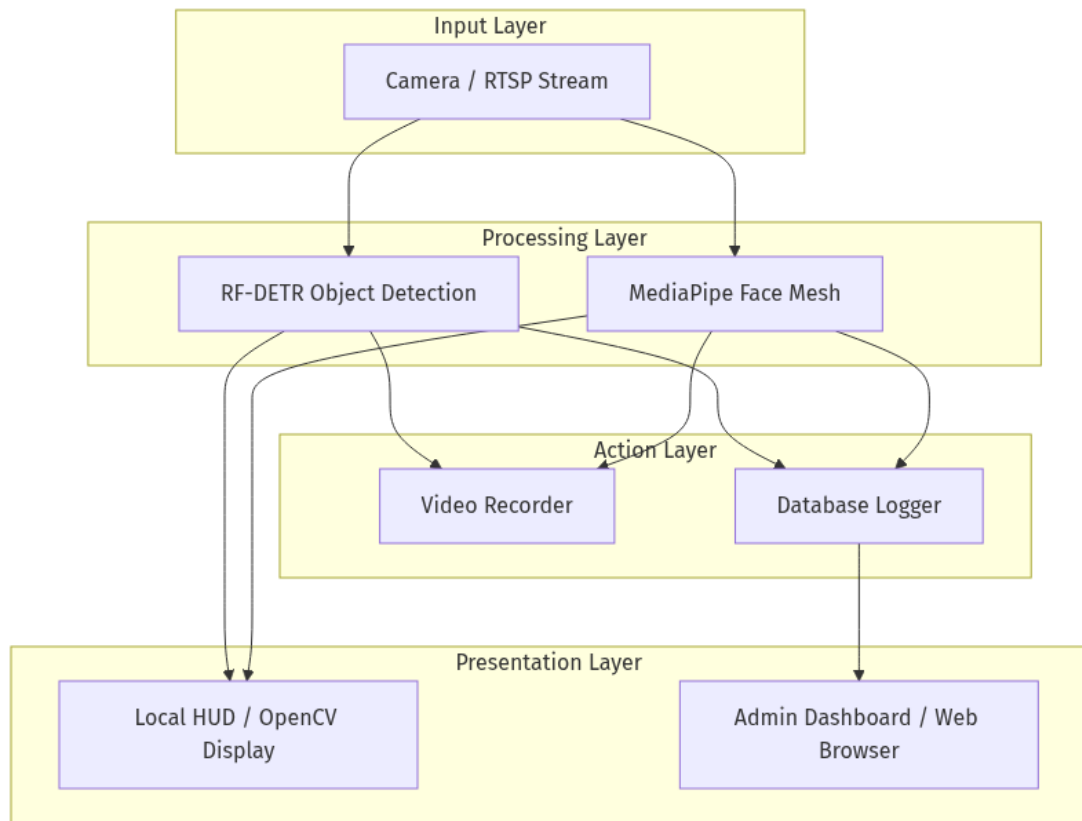


Figure 4.1: System Architecture

1. **Input Layer:** A local webcam or RTSP stream serves as the data acquisition interface.

2. **Processing Layer (AI):** The raw RGB frames are concurrently evaluated by the RF-DETR object detection model and the MediaPipe face mesh tracker.

3. **Action Layer:** When computational thresholds are breached, the system archives the screenshot or video clip while the Database Logger records the event metadata.

4. **Presentation Layer:** Outputs are displayed synchronously on a local OpenCV HUD and asynchronously on a Flask-based Admin Dashboard.

4.2.2 User Interface Design

The system utilizes two distinct interfaces: the local OpenCV HUD and the web-based Admin Dashboard.

Navigation Design: The Admin Dashboard operates as a Single Page Application (SPA). The navigation flow is controlled via a persistent Top Bar displaying live statistics and a collapsible Sidebar providing access to configuration settings.

Input Design:

Screens and Inputs: The local HUD accepts keyboard inputs (e.g., `[Q]` to quit, `[P]` to pause). The Admin Dashboard's settings screen includes toggle switches (selection inputs) to enable or disable specific AI modules, alongside numeric inputs and interactive sliders to dynamically adjust the `Confidence Threshold` and `Duration Threshold`.

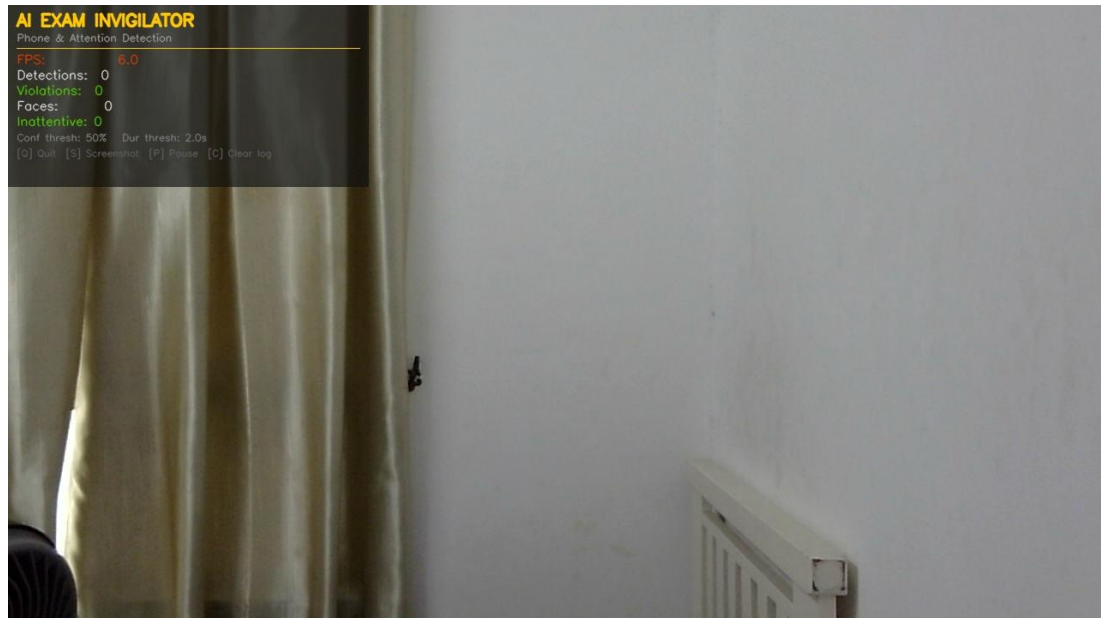


Figure 4.2: Local HUD Screen

Validation Rules: Numeric inputs for confidence thresholds are validated to ensure they strictly fall between `0.0` and `1.0`. Duration thresholds are validated to

ensure they are positive numeric values. Toggle switches are strictly validated as booleans.

The image shows a dark-themed admin interface with four main configuration sections, each with a dropdown header and a 'Reset to Defaults' button at the bottom.

- PHONE DETECTION**
 - Enable Module: ☒
 - Violation Duration (seconds): Slider at 0.5
 - Confidence Threshold: Slider at 0.75
- HEAD POSE DETECTION**
 - Enable Module: ☒
 - Violation Duration (seconds): Slider at 5
 - Yaw Threshold (degrees): Slider at 30
 - Pitch Down Threshold (degrees): Slider at 35
 - Pitch Up Threshold (degrees): Slider at 35
- EYE TRACKING**
 - Enable Module: ☒
 - Violation Duration (seconds): Slider at 5
 - Gaze Left Threshold: Slider at 0.35
- VIDEO RECORDING**
 - Pre-Violation Buffer (seconds): Slider at 4
 - Post-Violation Recording (seconds): Slider at 4

Reset to Defaults

Figure 4.3: Toggles and Slider in Admin Page

Output Design: Outputs are classified into two types:

Real-time Detail Reports: The local HUD overlays continuous bounding boxes and head-pose metrics on a continuous basis.

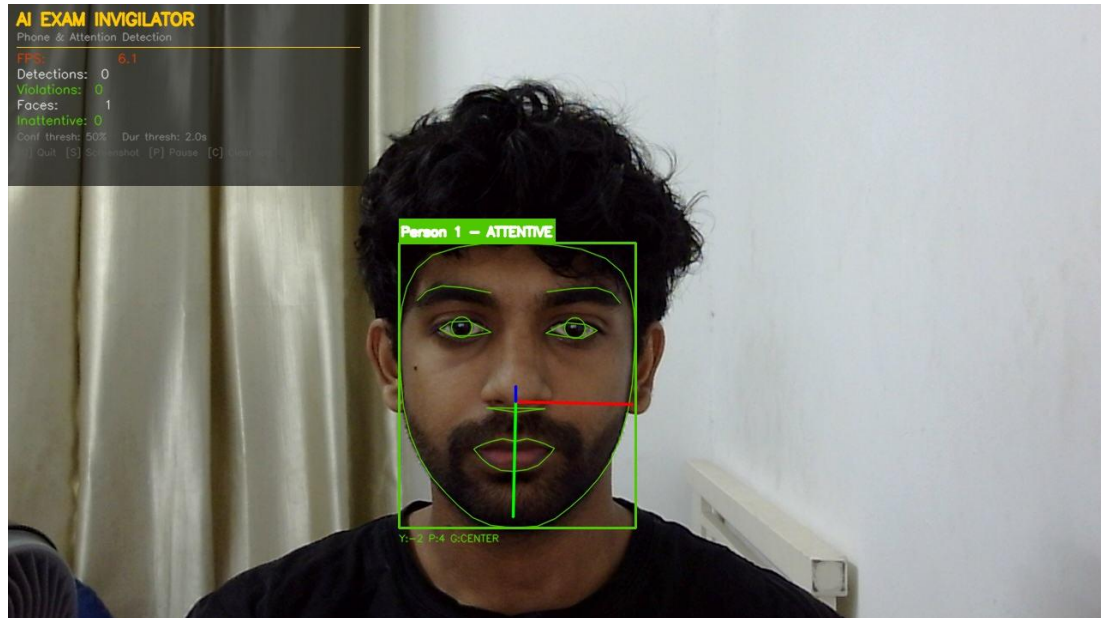


Figure 4.4: Real Time Overlay in HUD Display

Ad-hoc Summary Reports & Turnaround Documents: The Flask Dashboard outputs a grid of violation cards. Clicking a card opens a modal player or image viewer serving as an ad-hoc turnaround document to verify the recorded evidence.

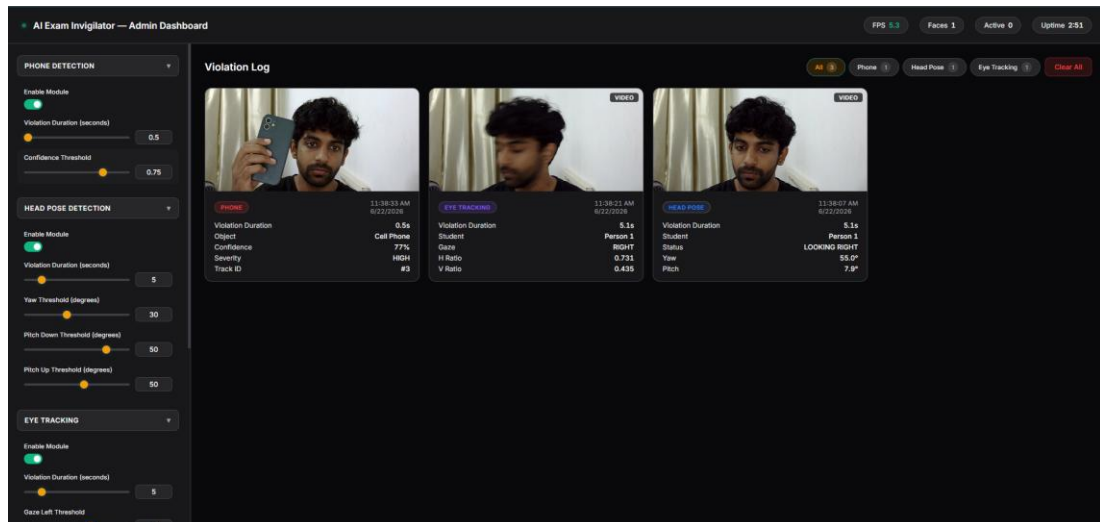


Figure 4.5: Admin Dashboard

4.2.3 Database Design

The database design utilizes a flat-table schema, operating via SQLite, to store event history efficiently without the overhead of complex relational mapping. The logical data model defines the structure for the primary entity:

Table 4.1

Violation			
int	id	PK	Auto incremented
datetime	timestamp		Date and time
string	type		Type of offense
float	confidence		AI confidence Score
string	media_path		Path to image or video

Data Dictionary & Normalization: The 'VIOLATION' table is normalized to First Normal Form (1NF) by ensuring each column contains atomic data (e.g., separating the media file path from the confidence score).

4.3 AI Component Design

Dataset: The RF-DETR model is pre-trained on the comprehensive COCO (Common Objects in Context) dataset. The system explicitly filters the neural network outputs to isolate specific class IDs relevant to cheating (e.g., "cell phone")

AI Techniques: The system employs the Real-time Detection Transformer (RF-DETR) for object boundary prediction and MediaPipe for Perspective-n-Point (PnP) facial landmark resolution.

Logic and Approach: The logic of the AI component is formulated to minimize false positives by requiring a temporal condition (duration) alongside the initial spatial condition (bounding box confidence or angular deviation).

The specific logic and functional flow of the AI approach is illustrated in the diagram below:

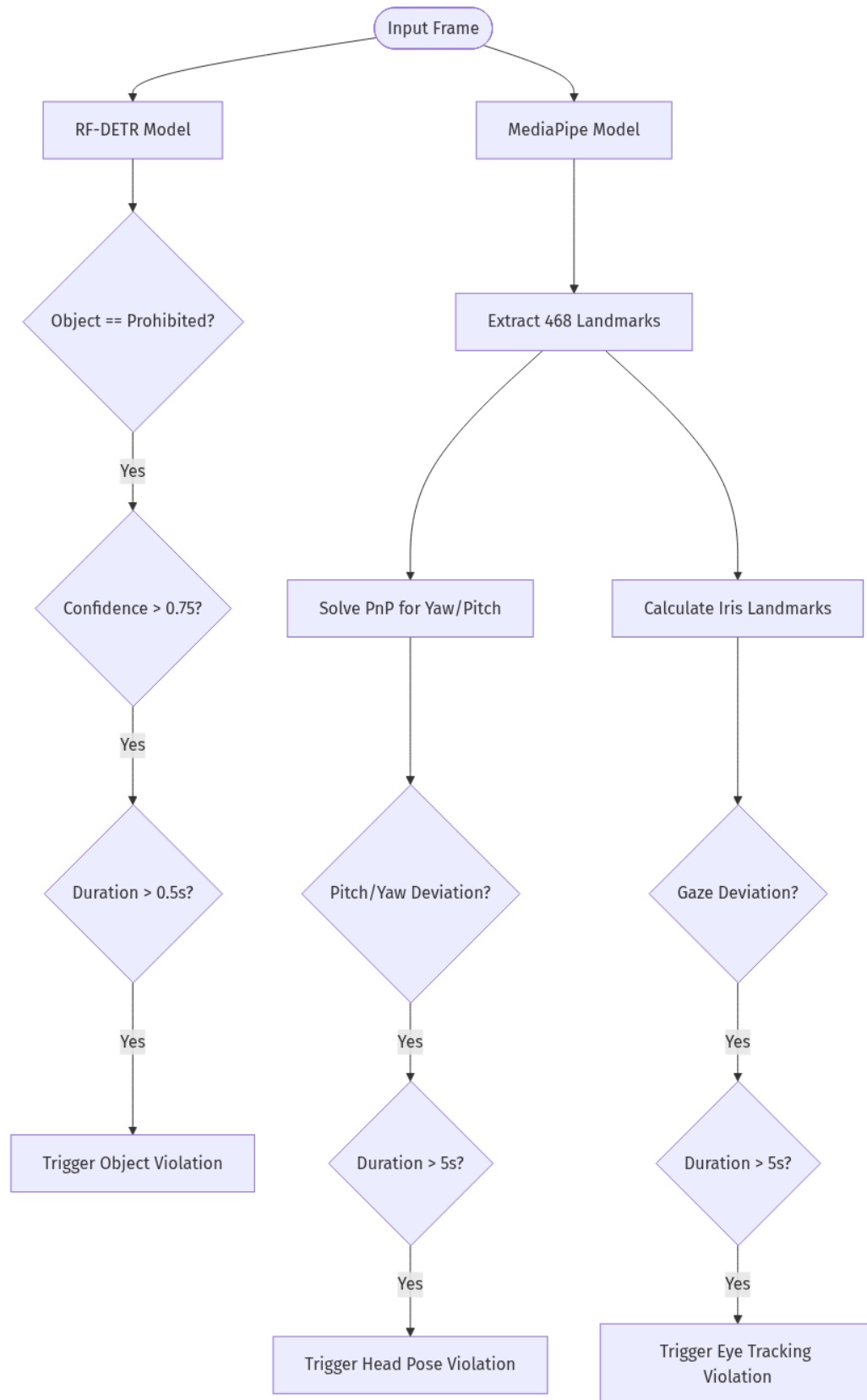


Figure 4.6: Flow of AI Component

4.4 Software or Hardware Design

Using the Prototyping software design cycle, the system's execution loop (`main.py`) acts as the orchestrator. In the initial stage, the models are loaded into RAM to avoid repetitive I/O overhead. Following this, an infinite pseudo-code loop executes:

1. Capture frame from buffer.
2. Pass frame matrix to RF-DETR and MediaPipe concurrently.
3. Compare resulting tensors against the threshold configurations.
4. If violation $>$ threshold limit, dispatch an asynchronous signal to save a screenshot or video clip.
5. Render the frame on the local UI matrix.

4.5 Summary

This chapter detailed the complete high-level design of the AI Exam Invigilator. It mapped out the system architecture, defined the UI, normalized the database schema, and detailed the logical flowchart governing the AI components. The next chapter will focus on the practical results and performance evaluation of these designs.

In the “Update Table of Contents” window, choose option “Update the entire table” and click OK. Microsoft Word will generate a new table of contents based on the student’s

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Introduction

This chapter presents the testing strategies employed and the evaluation of the AI techniques and functional requirements during the prototype testing phase. The testing phase involved two primary activities:

1. Evaluating the predictive performance of the underlying AI models (RF-DETR and MediaPipe) against their documented benchmarks and tailored behavioral testing.
2. Executing a Black-box testing strategy to validate the functional requirements of the complete system from an end-user perspective.

5.2 Evaluation of AI Techniques used in the project

- Object Detection: Based on the pre-trained COCO dataset evaluation, the RF-DETR-Nano model demonstrates high classification performance and exceptional speed, achieving an Average Precision (AP50) of 67.6% with an inference time of just 2.3 ms. In our specific behavioral testing, it successfully detected mobile phones even when partially occluded. False positives were strictly minimized by enforcing a minimum confidence threshold ($> 75\%$) and a time-duration requirement (> 0.5 continuous seconds).

- **Head Pose Tracking:** The MediaPipe Face Mesh model provides highly accurate 3D geometry with sub-millimeter Mean Absolute Error (MAE) under optimal lighting. The behavioral testing strategy involved a subject purposely turning their head away for extended periods. The system accurately flagged these events based on the pre-configured angular thresholds after 5 seconds.
- **Eye Tracking:** Iris landmarks were successfully utilized to determine the horizontal and vertical gaze ratios. The module accurately detected when a student was looking sideways without moving their head, triggering an alert after 5 continuous seconds of deviation.
- **System Performance:** On a standard multi-core CPU machine, the combined pipeline (RF-DETR + MediaPipe) maintained a steady framerate suitable for real-time monitoring (typically > 15 FPS), proving its robustness without requiring a dedicated GPU.

5.3 Testing of functional Requirement

The functional testing of the product was conducted using a Black-box approach. Below are the details of the test cases executed by the developer.

Test Case Id	TC-001
Tester Identification	System Developer
Action	User brings a mobile phone into the camera frame for more than 0.5 seconds.
Expected Result	HUD displays an alert, the database logs a "phone" violation, and a screenshot is saved locally.
Status	Success

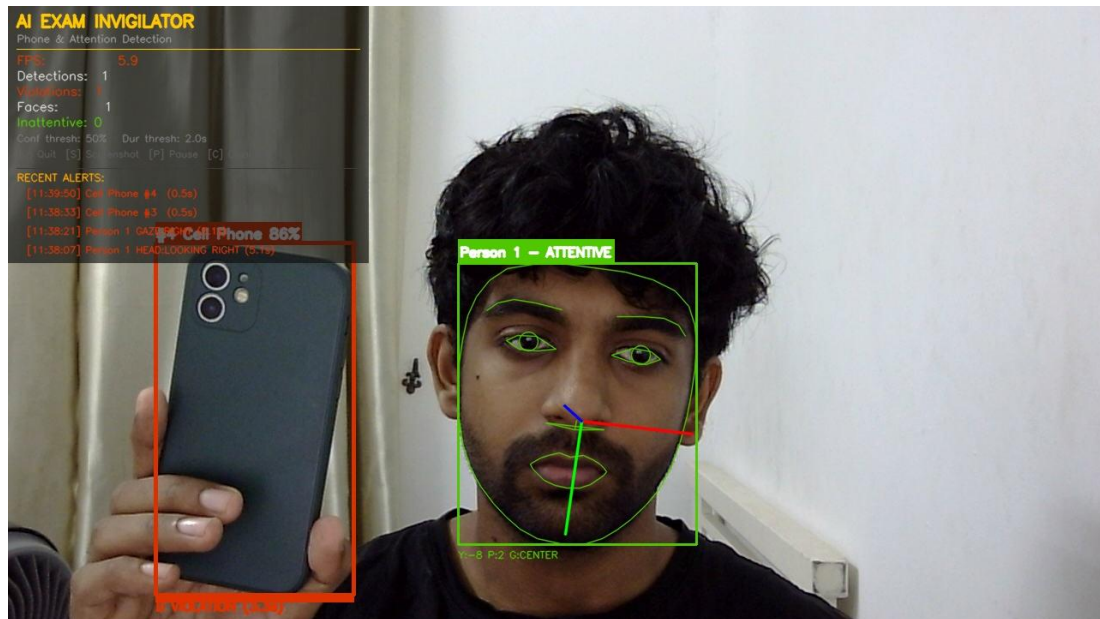


Figure 5.1: Test 1

Test Case Id	TC-002
Tester Identification	System Developer
Action	User looks down continuously for 5 seconds (simulating reading hidden notes).
Expected Result	HUD displays an "Attention Violation", the database logs the head pose event, and a video clip is saved.
Status	Success

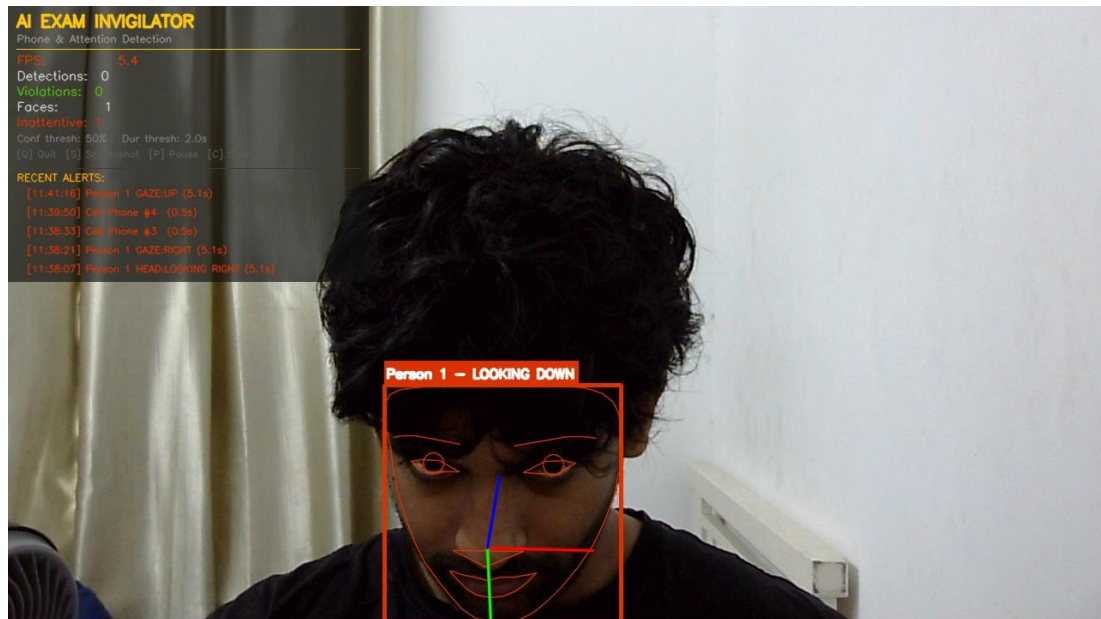


Figure 5.2: Test 2

Test Case Id	TC-003
Tester Identification	System Developer
Action	User looks sideways at a neighbor's desk without moving their head for 5 seconds.
Expected Result	HUD displays an "Eye Tracking Violation", the database logs the gaze event, and a video clip is saved.
Status	Success

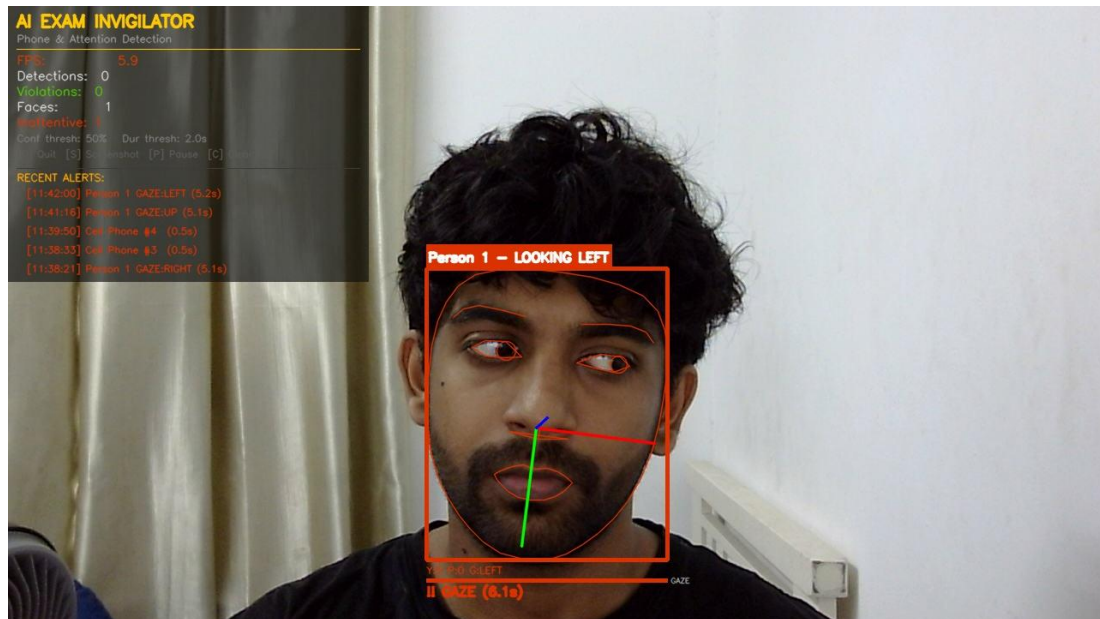


Figure 5.3: Test 3

Test Case Id	TC-004
Tester Identification	System Developer
Action	Open the Flask Admin Dashboard while a violation is occurring in real-time.
Expected Result	Live statistics update immediately, the new violation appears in the history table, and the evidence video clip is playable directly in the browser.
Status	Success

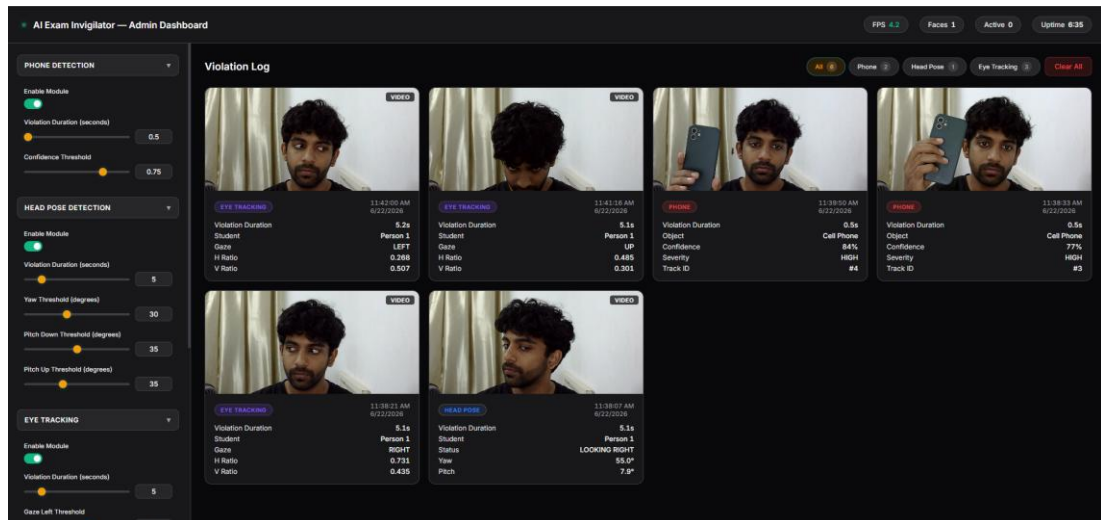


Figure 5.4: Test 4

5.4 Summary

The results confirm that the integration of the chosen AI models effectively fulfills the project objectives. The testing phase proved that the system's accuracy (mAP for detection, MAE for landmarks) and its functional robustness meet the criteria for a viable physical invigilation assistant. The next chapter will conclude the project and propose future improvements.

CHAPTER 6: CONCLUSION

6.1 Observation on Weakness and Strengths

Strengths: The system operates entirely locally, ensuring privacy and eliminating the need for expensive cloud AI processing. The automated evidence capture (screenshots and video clips) is highly efficient, saving invigilators from reviewing hours of footage.

Weakness: The system's accuracy is heavily dependent on camera placement. If a camera is placed too far away or at a severe angle, facial landmarks may be difficult to acquire. Detection can also be affected by poor lighting conditions.

6.2 Propositions for Improvement

1. **Multi-Camera Support:** The system could be improved by supporting simultaneous processing of multiple RTSP streams from different angles in the exam hall to cover blind spots.
2. **Audio Analysis:** Integrating a lightweight audio classification model could help detect whispering or suspicious noises during the exam.

6.3 Project Contribution

This project contributes a practical, localized AI solution for educational institutions. It provides a tangible tool (a complete software package with a dashboard) that can be deployed in physical examination halls to uphold academic integrity with minimal human effort.

6.4 Summary

The AI Exam Invigilator System successfully met its objectives. It demonstrates the viable application of state-of-the-art computer vision (RF-DETR and MediaPipe) in a real-world, physical proctoring scenario. The combination of real-time alerts and recorded evidence makes it a powerful asset for modern educational assessments.

REFERENCES

- Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. arXiv preprint arXiv:2004.10934.
- Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., ... & Grundmann, M. (2019). MediaPipe: A Framework for Building Perception Pipelines. arXiv preprint arXiv:1906.08172.
- Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Ding, Q., ... & Chen, J. (2023). DETRs Beat YOLOs on Real-time Object Detection. arXiv preprint arXiv:2304.08069.
- Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python. " O'Reilly Media, Inc."

APPENDICES